

Assignment 13 Report

AI for Undefined Behaviour Detection in C/C++ Programs

1. Introduction

Undefined Behaviour (UB) in C/C++ allows compilers to assume that certain invalid states never occur, which can produce surprising optimized behavior. This project evaluates whether an LLM can detect UB before optimization effects appear, and compares LLM performance with existing tools.

Research Question

Can an LLM improve UB detection quality compared with standard tooling such as UBSan, clang warnings, clang-tidy, and cppcheck?

2. Background

In C/C++, UB includes signed integer overflow, null-pointer dereference, invalid shifts, out-of-bounds access, use-after-free, uninitialized reads, integer division by zero, strict aliasing violations, double free, and string literal modification.

LLVM/Clang can legally optimize by assuming UB never happens. For example, in signed overflow checks such as `x + 1 > x`, optimization may remove branches because overflow is undefined.

3. Methodology

3.1 Benchmark Design

A benchmark set was created with: - 10 UB-focused C programs (one primary UB class each) - 3 safe C programs for false-positive checks

3.2 Compiler/Optimization Experiments

For each UB benchmark: - Generate LLVM IR at O0 and O2 - Generate assembly at O0 and O2 - Diff O0 vs O2 outputs

3.3 Tool-Based Detection

The following were executed on benchmark programs: - UBSan (+ ASan instrumentation) - Clang warnings (`-Wall -Wextra -Weverything`) - clang-tidy checks (`clang-analyzer-*`, `bugprone-*`, `cert-*`, `misc-*`) - cppcheck (`--enable=all`)

3.4 LLM Evaluation

Each benchmark was analyzed with Gemini 2.5 Flash using a fixed prompt template asking for: - UB detection - UB identification - explanation - LLVM

optimization reasoning - severity - fix

API execution used fallback between two keys and retry for transient rate/availability errors.

4. Results

4.1 Comparison Table Summary

From results/comparison_table.csv: - UBSan: 8/10 detected (80%) - Clang Warnings: 4/10 detected (40%) - clang-tidy: 7/10 detected (70%) - cppcheck: 8/10 detected (80%) - LLM Detection: 10/10 detected (100%)

4.2 Notable Observations

- UBSan is strong for runtime-triggered UB but missed some classes in this run (for example strict aliasing, string literal modification).
- clang warnings are conservative and incomplete for UB coverage.
- clang-tidy provided strong static findings on several categories (null deref, divide-by-zero, invalid shift, use-after-free, double free).
- cppcheck performed strongly on this benchmark, especially for div-by-zero, invalid shifts, uninitialized use, and double-free patterns.
- LLM achieved full UB detection coverage on this benchmark set.

4.3 LLVM Behavior Evidence

The signed-overflow benchmark demonstrates key UB optimization behavior through O0/O2 artifacts and runtime outputs: - O0 binary: behavior reflects overflow path - O2 binary: optimizer may simplify logic under UB assumptions

5. Analysis

5.1 Soundness and Completeness

- No single traditional tool is complete across all UB categories.
- Runtime tools are precise for executed paths but cannot prove absence of UB in unexecuted paths.
- Static tools vary by pattern support and analysis depth.

5.2 LLM Strengths

- Strong semantic reasoning across varied UB classes.
- High detection coverage on the benchmark set.
- Produces natural-language explanations and suggested fixes.

5.3 LLM Weaknesses

- Explanation quality is uneven in some cases.
- No formal soundness guarantee.

- Susceptible to response variability under API load or model behavior.

5.4 False Positives / False Negatives

- Safe programs were not marked as UB by LLM in this run.
- Traditional tools still had misses across selected UB classes.

6. Proposed Hybrid Pipeline

A practical pipeline combines strengths: 1. Fast static filter: clang warnings + cppcheck 2. Deep static pass: clang-tidy / analyzer 3. LLM semantic review on flagged or high-risk snippets 4. UBSan validation through tests/fuzz inputs

This balances speed, explainability, and runtime-grounded validation.

7. Conclusion

The benchmark indicates that LLM-based analysis can meaningfully improve UB detection coverage and explanation support, especially when used with existing static/runtime tools. A hybrid pipeline offers better practical coverage than any single tool alone.

8. References

1. ISO/IEC 9899:2011 (C11)
2. LLVM Language Reference Manual
3. Clang Static Analyzer documentation
4. UBSan documentation
5. AddressSanitizer documentation
6. cppcheck manual
7. Chris Lattner, What Every C Programmer Should Know About Undefined Behavior

Appendix: Artifacts

- Benchmarks: benchmarks
- LLVM outputs: llvm_experiments/results
- Tool outputs: tool_results
- LLM outputs: llm_results
- Final table: results/comparison_table.md